

DESENVOLVIMENTO DE SISTEMAS WEB I

Recapitulação Geral

Tudo que construímos até aqui — sem pressa, com espaço para perguntas

Antes de avançarmos para JavaScript (Aula 10)

Por que estamos parando aqui

- 1 Cobrimos muito conteúdo em pouco tempo — HTML, HTTP, MVC, Bootstrap, CRUD, banco de dados, Docker e REST.
- 2 É normal que partes tenham ficado soltas, principalmente as conexões ENTRE os assuntos.
- 3 Hoje não tem código novo — é uma aula de perguntas e respostas, tópico por tópico.
- 4 Vamos seguir junto o roteiro: recap curto de cada bloco, e daí abrimos para perguntas antes de seguir.

Linha do tempo do projeto

Aula 1

Fundamentos da web: Internet, HTTP, HTML, formulários

Aula 2

Ambiente de desenvolvimento: JDK, Maven, Tomcat

Aula 3

Arquitetura MVC — o padrão que sustenta tudo

Aula 4

Início do projeto oficial: Agenda + Bootstrap

Aula 5

CRUD: Read e Update, dados em memória

Aula 6

CRUD: Create e Delete, introdução a JDBC/PostgreSQL

Aula 7

Modelagem do banco com pgModeler

Aula 8

JDBC real: ConnectionFactory + DAO com SQL

Extra

PostgreSQL via Docker + pgAdmin

Aula 9

Criação da API REST (/api/contatos)

Fundamentos da Web

O QUE FIZEMOS

- Como a internet funciona: pacotes, IP, TCP garantindo entrega confiável
- HTTP: requisição (método, URL, headers, body) e resposta (status code)
- Navegadores: DOM, CSSOM, Layout e Paint
- HTML estrutural e semântico; formulários (action, method, name)

PERGUNTAS PARA A TURMA

1. O que acontece, passo a passo, entre digitar um endereço e a página aparecer na tela?
2. Qual a diferença entre GET e POST, e por que isso importa para os nossos formulários?
3. Por que usamos tags semânticas (header, main) em vez de só <div> para tudo?

Ambiente de Desenvolvimento

O QUE FIZEMOS

- JDK 25, Maven 3.9.16 e Apache Tomcat 11 instalados e configurados
- VSCode com Extension Pack for Java e Community Server Connectors
- Migração de javax.servlet (livros) para jakarta.servlet (Tomcat 11)

PERGUNTAS PARA A TURMA

1. Para que serve o Tomcat, exatamente? O que ele faz que o navegador sozinho não consegue?
2. Qual a diferença entre javax.servlet e jakarta.servlet, e por que isso importa no nosso código?
3. Para que serve o Maven no nosso projeto?

Arquitetura MVC

O QUE FIZEMOS

- Model (dados), View (JSP), Controller (Servlet) — cada um com uma única responsabilidade
- Fluxo: navegador → Controller → Model → Controller → forward → View
- `RequestDispatcher.forward()` mantém a mesma requisição do início ao fim
- MVC organiza código DENTRO de uma aplicação — diferente de microsserviços

PERGUNTAS PARA A TURMA

1. Com suas palavras: qual é o papel de cada letra do MVC no nosso projeto?
2. Por que o Controller nunca gera HTML diretamente?
3. Consegue imaginar um problema real que teríamos SEM essa separação?

O Projeto Agenda + Bootstrap

O QUE FIZEMOS

- Início do projeto oficial e contínuo do semestre
- Tela Inicial (index.jsp) e Agenda de Contatos (agenda.jsp)
- Bootstrap via CDN: classes prontas de botão, tabela e layout
- Estrutura de pastas: model, controller, dao, webapp

PERGUNTAS PARA A TURMA

1. Para que serve o Bootstrap? O que ele resolve que teríamos que fazer manualmente em CSS?
2. O que significam as classes btn-primary e table-striped?
3. Por que decidimos usar UM projeto único para o semestre inteiro, em vez de vários exercícios soltos?

CRUD: Read e Update

O QUE FIZEMOS

- ContatoDAO em memória — o problema do HTTP stateless recriando a lista a cada requisição
- Solução: `List<Contato> static`, sobrevivendo entre requisições
- Edição: o ID viaja de uma tela para outra via query string (`?id=`)
- Padrão Post/Redirect/Get: por que `sendRedirect()` depois de salvar, e não `forward()`

PERGUNTAS PARA A TURMA

1. Por que a lista de contatos "esquecia" as alterações quando era recriada dentro do `doGet`?
2. Como o ID do contato viaja da tela de listagem até a tela de edição?
3. O que aconteceria se, depois de salvar, déssemos F5 na página — com `forward()` e com `sendRedirect()`?

CRUD: Create e Delete

O QUE FIZEMOS

- NovoContatoServlet: formulário vazio, ID gerado manualmente (proximoid++)
- Exclusão via formulário POST + confirm() — nunca um link <a> simples
- Callback ao conceito de GET idempotente (Aula 1): excluir é um efeito colateral

PERGUNTAS PARA A TURMA

1. Por que a exclusão não deveria ser feita com um link <a> comum, em GET?
2. Hoje, quem decide o próximo ID de um contato novo?
3. O que significa dizer que o CRUD ficou "completo", mesmo ainda em memória?

Por que Banco de Dados + Modelagem

O QUE FIZEMOS

- Limitações da lista em memória: não persiste, não escala, não é assim que sistemas reais funcionam
- JDBC: a ponte entre Java e o banco (Driver, Connection, PreparedStatement, ResultSet)
- PostgreSQL escolhido: open-source, robusto, muito usado no mercado
- pgModeler: tabela contato modelada com PK, tipos e constraints (NOT NULL, UNIQUE), SQL gerado

PERGUNTAS PARA A TURMA

1. Quais são as limitações reais de guardar os dados só em uma List em memória?
2. O que é uma chave primária, e por que toda tabela precisa de uma?
3. Para que servem as constraints NOT NULL e UNIQUE — o que elas garantem que o Java sozinho não garante?

JDBC de Verdade: ConnectionFactory + DAO

O QUE FIZEMOS

- ConnectionFactory centraliza a criação de conexões (URL, usuário, senha em um só lugar)
- ContatoDAO reescrito inteiro com PreparedStatement e ResultSet
- "?" no SQL + setString()/setInt() protege contra SQL Injection
- try-with-resources fecha Connection, PreparedStatement e ResultSet automaticamente

PERGUNTAS PARA A TURMA

1. O que a ConnectionFactory resolve que não teríamos se cada método abrisse sua própria conexão?
2. Por que usamos "?" no SQL em vez de concatenar o valor direto na string?
3. O que o try-with-resources garante que um try comum não garantiria?

PostgreSQL via Docker + pgAdmin

O QUE FIZEMOS

- PostgreSQL rodando em um container Docker, isolado do sistema operacional
- docker run com variáveis de ambiente (usuário, senha, banco) e volume para persistir dados
- pgAdmin registrado como Server, conectando em localhost:5432

PERGUNTAS PARA A TURMA

1. Com suas palavras, o que é um container Docker?
2. Para que serve o volume (-v) no comando docker run? O que aconteceria sem ele?
3. O que muda, do ponto de vista do nosso projeto Java, quando o banco roda em um container?

Criação da API REST

O QUE FIZEMOS

- Diferença entre endpoints orientados a ação (/novo-contato) e orientados a recurso (/api/contatos)
- Verbos HTTP com significado semântico: GET, POST, PUT, DELETE
- ContatoRestServlet reaproveita 100% do ContatoDAO — só muda a "porta de entrada"
- JSON como formato de troca de dados; Gson para converter Java ↔ JSON

PERGUNTAS PARA A TURMA

1. Qual a diferença entre um endpoint como /novo-contato e um endpoint REST como /api/contatos?
2. Por que a mesma URI /api/contatos/{id} serve tanto para GET, quanto para PUT e DELETE?
3. Por que conseguimos criar a API REST sem reescrever nada do que já existia no projeto?

A arquitetura completa, hoje

VIEW

index.jsp, agenda.jsp, novo/editar-contato.jsp — HTML + Bootstrap + JSTL/EL

CONTROLLER (JSP)

AgendaServlet, NovoContatoServlet, EditarContatoServlet, ExcluirContatoServlet

CONTROLLER (REST)

ContatoRestServlet — GET / POST / PUT / DELETE em /api/contatos, devolvendo JSON

MODEL

Contato.java — id, nome, telefone, email

DAO

ContatoDAO — listar, buscarPorId, inserir, atualizar, excluir (JDBC)

BANCO DE DADOS

PostgreSQL 18 (via Docker) — tabela contato, modelada no pgModeler

Perguntas de síntese — conectando tudo

1 Consegue descrever, do início ao fim, o caminho completo de um clique em "Excluir" até o banco de dados e de volta para a tela?

2 O que muda, e o que NÃO muda, no projeto quando trocamos a fonte dos dados de uma List em memória para o PostgreSQL?

3 Por que conseguimos adicionar a API REST inteira sem reescrever nada do que já existia?

4 Se tivéssemos que explicar o projeto inteiro para alguém em 2 minutos, quais seriam os pontos mais importantes?

SEGUINDO EM FRENTE

Só avançamos quando o mapa geral fizer sentido.

Sem pressa — melhor visitar um bloco a mais hoje do que avançar com dúvidas acumuladas.

Próxima aula: Consumo de APIs REST com JavaScript (Aula 10)